Second-Order ADMM with Augmented Newton Steps

Avirup Mandal
Indian Institute of Technology Palakkad
Palakkad, Kerala, India

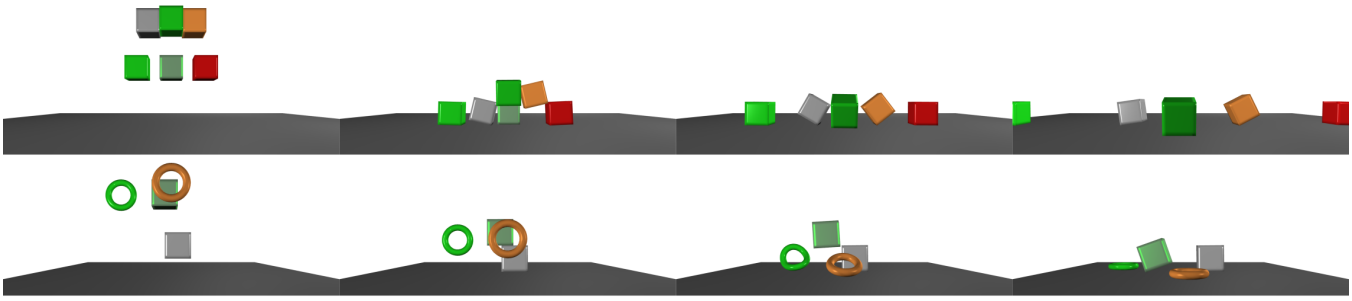


Figure 1: Simulation of various deformable objects using ADMM augmented with a Newton step. (Top) Six box-shaped objects with linear hyperelastic energies are dropped onto a rigid surface. They collide with the surface and with one another. (Bottom) Two tori and two boxes, each with different elastic properties, fall onto a rigid sheet, rebound and collide with one another.

Abstract

We present an augmented alternating direction method of multipliers (ADMM) framework enhanced with Newton steps, combining the robustness and flexibility of ADMM with the fast local convergence of second-order optimization. We investigate the conditions under which this augmentation is applicable while retaining its real-time simulation speed. The proposed method applies ADMM with an augmented Newton step to the implicit time integration of elastic and deformable bodies, enabling an efficient solution to large-scale constrained optimization problems arising in dynamic simulation. Numerical experiments demonstrate that the proposed method significantly reduces the number of iterations, overall runtime, and improves accuracy compared to the state-of-the-art ADMM algorithm. We validate the method's effectiveness on a variety of challenging examples, including volumetric deformable bodies with various nonlinear hyper-elastic energies.

Keywords

physics-based simulation, newton optimization, dynamics

1 Introduction

Simulating physically realistic deformable objects at interactive rates has long been a central goal of computer graphics research. Researchers developed advanced techniques capable of generating highly realistic animations of deformable objects. In computer graphics, substantial research has focused on deriving the physical properties of soft tissues, such as muscle and rubber, using a constitutive modelling approach [53] [22] [44]. While these methods can produce highly realistic simulations, they are computationally expensive. As a result, a separate line of research prioritizes real-time rendering, trading some physical accuracy for efficiency. Position-based dynamics [37] [2], projective dynamics [5], and alternating direction method of multipliers (ADMM)-based approaches [41] exemplify this category, offering parallelizable, robust, and fast solutions. ADMM approaches an optimization problem by iteratively solving two coupled subproblems subject to soft constraints.

In this work, we extend the ADMM framework by incorporating a Newton optimization step. Traditional ADMM alternates between solving global and local subproblems to minimize the objective function, relying solely on first-order gradient information for the soft constraints. The objective function includes a hyperelastic strain-energy term to model the behavior of deformable objects. However, standard ADMM does not exploit second-order information because it does not use the Hessian of the cost function during optimization. Our goal is to integrate a Newton step into the ADMM process to leverage second-order information, thereby accelerating convergence and reducing the number of iterations required. We further describe techniques for applying a Newton-step–augmented ADMM framework to deformable object simulation while maintaining real-time performance. This is achieved by approximating the inverse Hessian of the Newton-step augmented ADMM formulation at the start of the simulation and keeping it constant throughout the optimization process. The major contributions of our work are as follows: *We propose a novel Newton-augmented ADMM formulation that accelerates the numerical solution of deformable-body dynamics, enabling efficient, real-time simulation of deformable objects.* We validate the effectiveness of the proposed method on a variety of challenging examples, including volumetric deformable bodies governed by different nonlinear hyperelastic energy models. We compare our approach with ADMM and demonstrate superior performance, with faster convergence and reduced runtime.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 describes the technical details of the proposed method. Section 4 presents the experimental results. Finally, Section 5 concludes the paper by discussing the limitations of this work and outlining directions for future research.

2 Related Works

Many deformable materials of interest in computer graphics—such as cloth, rubber, muscle, and biological tissue—exhibit strongly nonlinear elastic behavior, particularly under large deformations. A substantial body of prior work has therefore focused on capturing and modeling such nonlinear constitutive responses. Data-driven approaches have been proposed for volumetric soft tissues [3] [36] and cloth materials [49] [35], demonstrating that nonlinear elasticity yields significantly more realistic visual behavior than linear elastic models. Beyond physical realism, nonlinear elasticity also enables greater artistic control — Xu et al. [53] showed that allowing artists to specify nonlinear stress–strain responses directly can produce desirable and expressive animation effects. A new approach to designing sigularity-free hyperelastic energy using eigenanalysis is discussed in a series of subsequent works by Smith et al. [43], Smith et al. [44], Kim et al. [21], and Kim and Eberle [22].

Implicit time-integration schemes, such as the backward Euler method, have long been favored in physics-based animation due to their robustness and unconditional stability [46] [1]. However, their practical application is often limited by the computational cost of solving large nonlinear systems, especially when multiple Newton iterations are required. Simpler iterative solvers, such as Jacobi or Gauss–Seidel methods, typically converge too slowly for interactive use.

To address these limitations, several fast and parallel implicit integration techniques have been proposed in recent years. Many of these methods exploit the optimization-based formulation of the backward Euler method [20] [34] [12], alternating between local and global update steps. Liu et al. [28] introduced a block-coordinate descent method for mass-spring systems, while Bouaziz et al. [5] generalized this paradigm to finite elements and constrained dynamics within the projective dynamics framework. Related ideas also appear in Shape-Up [4], which uses constraint projections for geometry processing. Galerkin multigrid methods use massive GPU parallelization for the interactive simulation of deformable bodies [52] [31]. Extensions of projective dynamics have explored broader applications and improved performance. Weiler et al. [50] adapted the framework to incompressible fluids by employing a parallel, matrix-free conjugate-gradient solver. Fratarcangeli et al. [11] proposed the Vivace solver, employing graph-colored Gauss–Seidel iterations to enable GPU acceleration and dynamic constraint handling. However, these approaches generally rely on simplified elastic models in which forces are proportional to distances from constraint manifolds. Several recent works [7, 23, 30] also investigated efficient real-time simulation of hyperelastic objects.

Position-based dynamics (PBD) [37] [2] is another influential class of methods, treating all forces as constraints and resolving them via Gauss–Seidel iterations. XPBD [32] extends this framework by introducing compliance, allowing constraints to represent finite stiffness. Wang [48] proposed Chebyshev semi-iterative acceleration for both projective dynamics and PBD. Liu et al. [29] interpreted projective dynamics as a quasi-Newton method and extended it to nonlinear materials, further accelerating convergence using L-BFGS [39]. Li et al. [25] present an interactive approach for cloth cutting and tearing based on Projective Dynamics. Kee et al. [19] extended the original projective dynamics technique by adding a novel energy-momentum conserving constraint. Overby et al. [41] proved that projective dynamics is a special case of a more general optimization algorithm named ADMM. Inexact Newton ADMM techniques are proposed for classification [9] and multi-agent optimization tasks [27].

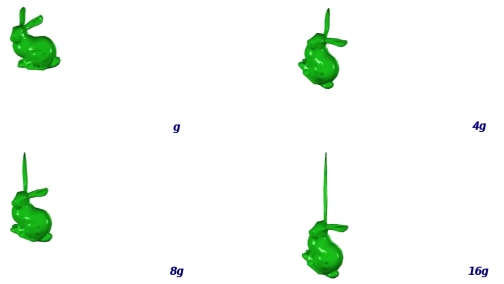


Figure 2: A bunny hangs under varying gravitational forces.

In the optimization literature, primal–dual methods have gained prominence for their scalability and decomposition properties. Algorithms such as the alternating direction method of multipliers (ADMM) [6], the alternating minimization algorithm (AMA) [47], and the primal–dual hybrid gradient (PDHG) method [55] decompose objective functions into simpler subproblems that can be optimized alternately. This structure is particularly advantageous when individual terms are easy to handle in isolation but difficult to optimize jointly.

Although ADMM was initially developed for convex optimization, it has proven effective in many nonconvex settings, including distributed clustering [10], phase retrieval [51], and nonnegative matrix factorization [45]. Recent theoretical work has established convergence guarantees for certain classes of nonconvex problems [17] [24] [33]. Accelerated variants of ADMM have also been proposed for problems with additional regularity [14] [15] [18]. However, none of these methods uses a second-order Hessian update. Zhang et al. [54] uses Anderson-type extrapolation on ADMM iterations to improve convergence and speed. The authors extended the work for non-convex optimization problems [40]. Similarly, Overby et al. [42] uses ADMM with specially designed global-injectivity constraints/energies and non-injective intermediate steps for geometric optimization, and Li et al. [26] decomposes large-step elastodynamic optimization/integration into subproblems to obtain efficient simulation. However, the proposed methods do not convert the ADMM subproblem into a Newton-based second-order solve, thereby completely ignoring the curvature information in the optimization.

Primal–dual methods have also begun to appear in graphics applications. Gregson et al. [16] used ADMM to enforce incompressibility constraints in fluid tracking. The augmented Lagrangian method, closely related to ADMM, has been applied to strain-limiting in cloth [38]. Later, ADMM is extended to deformable object [41] and hair [8] simulation. Despite their success, none of the ADMM techniques use the Hessian of the objective function, thereby omitting curvature information. Our goal is to incorporate the Hessian via a Newton step to improve the convergence rate and accuracy.



Figure 3: Three beams with Neo-Hookean, StVK, and co-rotational hyperelastic energies (top to bottom) are stretched.

3 Methodology

In this section, we first introduce basic notions of deformable body simulation and optimization methods, such as ADMM and the Newton method. Later, we introduce the augmented Newton steps-ADMM technique.

3.1 Implicit Integration for Deformable Simulation

Let's consider a physical system with position $\boldsymbol{x} \in \mathbb{R}^d$, velocities $\boldsymbol{v} \in \mathbb{R}^d$, and mass matrix M. Given an external force $\boldsymbol{f}^{ext}$, the implicit Euler integration yields the following system dynamics [22]

$$\begin{bmatrix} \boldsymbol{x}_{t+\Delta t} - \boldsymbol{x}_t \\ \boldsymbol{v}_{t+\Delta t} - \boldsymbol{v}_t \end{bmatrix} = \Delta t \begin{bmatrix} \boldsymbol{v}_{t+\Delta t} \\ M^{-1}\left(\boldsymbol{f}^{ext}_t + \boldsymbol{f}^{int}_{t+\Delta t}\right) \end{bmatrix} \tag{1}$$

Using Taylor Series expansion, we can rearrange the terms of Equation 1 to solve for the position and velocity.

$$\left[M - \Delta t \frac{\partial \boldsymbol{f}^{int}}{\partial \boldsymbol{v}} - \Delta t^2 \frac{\partial \boldsymbol{f}^{int}}{\partial \boldsymbol{x}}\right] \Delta \boldsymbol{v} = \Delta t \left[\boldsymbol{f}^{ext} + \boldsymbol{f}^{int} + \Delta t \frac{\partial \boldsymbol{f}^{int}}{\partial \boldsymbol{x}} \boldsymbol{v}_t\right] \tag{2}$$

$$\left[M - \Delta t \frac{\partial \boldsymbol{f}^{int}}{\partial \boldsymbol{v}} - \Delta t^2 \frac{\partial \boldsymbol{f}^{int}}{\partial \boldsymbol{x}}\right] \Delta \boldsymbol{x} = \Delta t^2 \left(\boldsymbol{f}^{ext} + \boldsymbol{f}^{int}\right) + \Delta t \left[M - \Delta t \frac{\partial \boldsymbol{f}^{int}}{\partial \boldsymbol{v}}\right] \boldsymbol{v}_t \tag{3}$$

This strategy to linearize the system about the current state follows Newton's method in optimization. Consider minimizing a twice differentiable function $h : \mathbb{R}^n \longrightarrow \mathbb{R}$. The iterative update for Newton's optimization method is as follows.

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \left[\nabla^2_{\boldsymbol{xx}} h(\boldsymbol{x}_k)\right]^{-1} \nabla_{\boldsymbol{x}} h(\boldsymbol{x}_k) \tag{4}$$

where $\boldsymbol{x}_k$ is the parameter value at the kth iteration at a particular time step. However, when $\boldsymbol{f}^{int}$ is strongly nonlinear, the optimization process requires multiple Newton solves, increasing the computation time.

In deformable object simulation, the internal forces, $\boldsymbol{f}^{int}$, are conservative and expressed as the gradient of hyper-elastic potential energy $\boldsymbol{f}^{int} = -V\nabla_{\boldsymbol{x}}\Psi(\boldsymbol{x})$, where V denotes volume. In such a case, Equation 3 can be reformulated as the following optimization problem

$$\boldsymbol{x}_{t+\Delta t} = \underset{\boldsymbol{x}}{\operatorname{argmin}} \ \frac{1}{2\Delta t^2} ||\boldsymbol{x} - \tilde{\boldsymbol{x}}_{t+\Delta t}||^2_M + \Psi(\boldsymbol{x}) \tag{5}$$

where $||\boldsymbol{x}||_M = \sqrt{\boldsymbol{x}^\top M \boldsymbol{x}}$ and $\tilde{\boldsymbol{x}}_{t+\Delta t} = \boldsymbol{x}_t + \boldsymbol{v}_t \Delta t + M^{-1}\boldsymbol{f}^{ext}\Delta t^2$ is the predicted position. The potential energy density, $\Psi(\boldsymbol{x})$, is the sum of local internal energy density terms, such as energy for each element in Finite Element Analysis.

$$\Psi(\boldsymbol{x}) = \sum_{i=1}^{m} \Psi_i(D_i \boldsymbol{x}) = \Psi_*(D\boldsymbol{x}), \quad \forall i \in \{1, 2, \dots m\} \tag{6}$$

where $D = \begin{bmatrix} D_1^\top & D_2^\top & \dots & D_m^\top \end{bmatrix}^\top$ and each D_i is a reduction matrix that only depends on a small local subset of nodes. Given the special structure of the problem, Overby et al. [41] employed ADMM algorithm in order to solve it efficiently. For further details, interested readers are referred to the works of Kim and Eberle [22] Overby et al. [41]. Our key objective is to improve ADMM for deformable object simulation by incorporating a Newton step. To that end, we first provide background on ADMM.

3.2 ADMM

The alternating direction method of multipliers (ADMM) [6] is an optimization method to solve a 2-block separable convex problem in the following form

$$\min_{\boldsymbol{x},\boldsymbol{z}} \ f(\boldsymbol{x}) + g(\boldsymbol{z}), \quad \text{s.t.} \ \ A\boldsymbol{x} + B\boldsymbol{z} = \boldsymbol{c} \tag{7}$$

where $f : \mathbb{R}^n \longrightarrow \mathbb{R}$ and $g : \mathbb{R}^m \longrightarrow \mathbb{R}$ are proper functions with the other parameters defined as $\boldsymbol{x} \in \mathbb{R}^n$, $\boldsymbol{z} \in \mathbb{R}^m$, $A \in \mathbb{R}^{p\times n}$, $B \in \mathbb{R}^{p\times m}$, and $c \in \mathbb{R}^p$.

The ADMM algorithm solves the separable two-block problem using an augmented Lagrange function [13]

$$\mathcal{L}_\rho(\boldsymbol{x}, \boldsymbol{z}, \boldsymbol{\lambda}) = f(\boldsymbol{x}) + g(\boldsymbol{z}) + \boldsymbol{\lambda}^\top (A\boldsymbol{x} + B\boldsymbol{z} - \boldsymbol{c}) + \frac{\rho}{2} ||A\boldsymbol{x} + B\boldsymbol{z} - \boldsymbol{c}||_2^2 \tag{8}$$

where λ denotes the Lagrange multiplier associated with the equality constraints, and $\rho > 0$ is a step-size parameter. The algorithm proceeds by iteratively applying the following updates until convergence.

$$\begin{aligned} \boldsymbol{x}_{k+1} &= \underset{\boldsymbol{x}}{\operatorname{argmin}} \ \mathcal{L}_\rho(\boldsymbol{x}, \boldsymbol{z}_k, \boldsymbol{\lambda}_k) \\ \boldsymbol{z}_{k+1} &= \underset{\boldsymbol{z}}{\operatorname{argmin}} \ \mathcal{L}_\rho(\boldsymbol{x}_{k+1}, \boldsymbol{z}, \boldsymbol{\lambda}_k) \\ \boldsymbol{\lambda}_{k+1} &= \boldsymbol{\lambda}_k + \rho\left(A\boldsymbol{x}_{k+1} + B\boldsymbol{z}_{k+1} - \boldsymbol{c}\right) \end{aligned} \tag{9}$$

If f and g are closed, convex, and proper functions, ADMM is proved to be convergent.

ADMM is invariant to rescaling the optimization variables $\boldsymbol{x}$ and $\boldsymbol{z}$. In particular, consider the change of variables $\hat{\boldsymbol{x}} = P\boldsymbol{x}$ and $\hat{\boldsymbol{z}} = Q\boldsymbol{z}$, where P and Q are invertible matrices. Applying ADMM to the resulting transformed problem, with objective functions $f(P^{-1}\hat{\boldsymbol{x}})$ and $g(Q^{-1}\hat{\boldsymbol{z}})$, yields

$$\min_{\boldsymbol{x},\boldsymbol{z}} \ f(P^{-1}\boldsymbol{x}) + g(Q^{-1}\boldsymbol{z}), \quad \text{s.t.} \ \ AP^{-1}\boldsymbol{x} + BQ^{-1}\boldsymbol{z} = \boldsymbol{c} \tag{10}$$

The transformed problem also requires the same sequence of iterations as the original one. However, ADMM is not invariant to rescaling the linear constraint

$$\min_{\boldsymbol{x},\boldsymbol{z}} \ f(\boldsymbol{x}) + g(\boldsymbol{z}), \quad \text{s.t.} \ \ WA\boldsymbol{x} + WB\boldsymbol{z} = W\boldsymbol{c} \tag{11}$$

For some invertible matrix W, it significantly affects the iterates and the convergence rate. In their work, Overby et al. [41] showed that appropriately choosing W leads to faster convergence. We now discuss how to improve ADMM by augmenting it with a Newton step update.



Figure 4: Two boxes fell on a sheet under gravity and rest on top of one another.



Figure 5: A torus falls onto a rigid sheet and rebounds.

3.3 ADMM with Augmented Newton Step

Our objective is to accelerate the convergence of the ADMM algorithm by reducing the number of iteration steps and thus the overall simulation runtime. Newton's method converges more rapidly because each iteration incorporates both gradient and curvature (second-order) information. So, we aim to update the original ADMM algorithm with an augmented Newton step. To employ the Newton step update, it is necessary first to compute the gradient and Hessian of the Lagrangian dual function [9] [27].

$$d(\boldsymbol{\lambda}) = \min_{\boldsymbol{x},\boldsymbol{z}} \ \mathcal{L}_\rho(\boldsymbol{x}(\boldsymbol{\lambda}), \boldsymbol{z}(\boldsymbol{\lambda}), \boldsymbol{\lambda}) \tag{12}$$

The gradient of the dual function, $d(\boldsymbol{\lambda})$, w.r.t. $\boldsymbol{\lambda}$ is given by

$$\nabla_{\boldsymbol{\lambda}} d(\boldsymbol{\lambda}) = \nabla_{\boldsymbol{\lambda}} \mathcal{L}_\rho(\boldsymbol{x}(\boldsymbol{\lambda}), \boldsymbol{z}(\boldsymbol{\lambda}), \boldsymbol{\lambda}) = A\boldsymbol{x}(\boldsymbol{\lambda}) + B\boldsymbol{z}(\boldsymbol{\lambda}) - \boldsymbol{c} \tag{13}$$

To compute the Hessian, differentiate the gradient again w.r.t. $\boldsymbol{\lambda}$

$$\nabla^2_{\boldsymbol{\lambda}\boldsymbol{\lambda}} d(\boldsymbol{\lambda}) = A\frac{d\boldsymbol{x}(\boldsymbol{\lambda})}{d\boldsymbol{\lambda}} + B\frac{d\boldsymbol{z}(\boldsymbol{\lambda})}{d\boldsymbol{\lambda}} = \begin{bmatrix} A & B \end{bmatrix} \begin{bmatrix} \frac{d\boldsymbol{x}}{d\boldsymbol{\lambda}} \\ \frac{d\boldsymbol{z}}{d\boldsymbol{\lambda}} \end{bmatrix} \tag{14}$$

From Equation 12, the first-order optimality conditions of the augmented Lagrangian minimization are

$$\begin{aligned} \nabla_{\boldsymbol{x}} f(\boldsymbol{x}) + A^\top \boldsymbol{\lambda} + \rho A^\top (A\boldsymbol{x} + B\boldsymbol{z} - \boldsymbol{c}) = 0 \\ \nabla_{\boldsymbol{z}} g(\boldsymbol{z}) + B^\top \boldsymbol{\lambda} + \rho B^\top (A\boldsymbol{x} + B\boldsymbol{z} - \boldsymbol{c}) = 0 \end{aligned} \tag{15}$$

Let's assume the KKT operator $\mathcal{K}$ as

$$\mathcal{K}(\boldsymbol{x}(\boldsymbol{\lambda}), \boldsymbol{z}(\boldsymbol{\lambda}), \boldsymbol{\lambda}) = \begin{bmatrix} \nabla_{\boldsymbol{x}} f(\boldsymbol{x}) + \rho A^\top (A\boldsymbol{x} + B\boldsymbol{z} - \boldsymbol{c}) \\ \nabla_{\boldsymbol{z}} g(\boldsymbol{z}) + \rho B^\top (A\boldsymbol{x} + B\boldsymbol{z} - \boldsymbol{c}) \end{bmatrix} + \begin{bmatrix} A^\top \boldsymbol{\lambda} \\ B^\top \boldsymbol{\lambda} \end{bmatrix} \tag{16}$$

At optimality $\mathcal{K}(\boldsymbol{x}^*(\boldsymbol{\lambda}), \boldsymbol{z}^*(\boldsymbol{\lambda}), \boldsymbol{\lambda}) = 0$. Differentiating $\mathcal{K}(\boldsymbol{x}(\boldsymbol{\lambda}), \boldsymbol{z}(\boldsymbol{\lambda}), \boldsymbol{\lambda})$ with $\boldsymbol{\lambda}$ at the optimal points $\boldsymbol{x}^*$ and $\boldsymbol{z}^*$, we get

$$\begin{bmatrix} \nabla^2_{\boldsymbol{x}\boldsymbol{x}} f(\boldsymbol{x}^*) + \rho A^\top A & \rho A^\top B \\ \rho B^\top A & \nabla^2_{\boldsymbol{z}\boldsymbol{z}} g(\boldsymbol{z}^*) + \rho B^\top B \end{bmatrix} \begin{bmatrix} \frac{d\boldsymbol{x}^*}{d\boldsymbol{\lambda}} \\ \frac{d\boldsymbol{z}^*}{d\boldsymbol{\lambda}} \end{bmatrix} + \begin{bmatrix} A^\top \\ B^\top \end{bmatrix} = 0 \tag{17}$$

Hence,

$$\begin{bmatrix} \frac{d\boldsymbol{x}^*}{d\boldsymbol{\lambda}} \\ \frac{d\boldsymbol{z}^*}{d\boldsymbol{\lambda}} \end{bmatrix} = -\begin{bmatrix} \nabla^2_{\boldsymbol{x}\boldsymbol{x}} f(\boldsymbol{x}^*) + \rho A^\top A & \rho A^\top B \\ \rho B^\top A & \nabla^2_{\boldsymbol{z}\boldsymbol{z}} g(\boldsymbol{z}^*) + \rho B^\top B \end{bmatrix}^{-1} \begin{bmatrix} A^\top \\ B^\top \end{bmatrix} \tag{18}$$

Substituting the values of $\frac{d\boldsymbol{x}^*}{d\boldsymbol{\lambda}}$ and $\frac{d\boldsymbol{z}^*}{d\boldsymbol{\lambda}}$ into Equation 14, the dual function, yields

$$\nabla^2_{\boldsymbol{\lambda}\boldsymbol{\lambda}} d(\boldsymbol{\lambda}) = -\begin{bmatrix} A & B \end{bmatrix} \begin{bmatrix} \nabla^2_{\boldsymbol{x}\boldsymbol{x}} f(\boldsymbol{x}^*) + \rho A^\top A & \rho A^\top B \\ \rho B^\top A & \nabla^2_{\boldsymbol{z}\boldsymbol{z}} g(\boldsymbol{z}^*) + \rho B^\top B \end{bmatrix}^{-1} \begin{bmatrix} A^\top \\ B^\top \end{bmatrix} \tag{19}$$

Following the arguments explained above, the original iterative update steps of ADMM can be reformulated with an augmented Newton step algorithm as

$$\begin{aligned} \boldsymbol{x}_{k+1} &= \operatorname*{argmin}_{\boldsymbol{x}} \ \mathcal{L}_\rho(\boldsymbol{x}, \boldsymbol{z}_k, \boldsymbol{\lambda}_k) \\ \boldsymbol{z}_{k+1} &= \operatorname*{argmin}_{\boldsymbol{z}} \ \mathcal{L}_\rho(\boldsymbol{x}_{k+1}, \boldsymbol{z}, \boldsymbol{\lambda}_k) \\ \boldsymbol{\lambda}_{k+1} &= \boldsymbol{\lambda}_k + H(\boldsymbol{x}_{k+1}, \boldsymbol{z}_{k+1}, \boldsymbol{\lambda}_k)^{-1} (A\boldsymbol{x}_{k+1} + B\boldsymbol{z}_{k+1} - \boldsymbol{c}) \end{aligned} \tag{20}$$

where

$$H(\boldsymbol{x}_{k+1}, \boldsymbol{z}_{k+1}, \boldsymbol{\lambda}_k) = \begin{bmatrix} A & B \end{bmatrix} \begin{bmatrix} \nabla^2_{\boldsymbol{x}\boldsymbol{x}} f(\boldsymbol{x}_{k+1}) + \rho A^\top A & \rho A^\top B \\ \rho B^\top A & \nabla^2_{\boldsymbol{z}\boldsymbol{z}} g(\boldsymbol{z}_{k+1}) + \rho B^\top B \end{bmatrix}^{-1} \begin{bmatrix} A^\top \\ B^\top \end{bmatrix} \tag{21}$$

In the following section, we examine how this method is applied to the simulation of deformable objects, detailing the process and associated considerations.

3.4 Implicit Integration for Deformable Simulation with Newton Augmented ADMM

As noted earlier, our objective is to solve the optimization problem in Equation 5 via implicit integration using an augmented ADMM algorithm with a Newton update. Using Equation 7, the optimization problem from Equation 5 is reformulated as

$$\min_{\boldsymbol{x},\boldsymbol{z}} \ \frac{1}{2\Delta t^2} ||\boldsymbol{x} - \tilde{\boldsymbol{x}}||^2_M + \Psi_*(\boldsymbol{z}), \quad \text{s.t.} \ \ W(D\boldsymbol{x} - \boldsymbol{z}) = 0 \tag{22}$$

where

$$\begin{aligned} f(\boldsymbol{x}) = \frac{1}{2\Delta t^2} ||\boldsymbol{x} - \tilde{\boldsymbol{x}}||^2_M, \quad g(\boldsymbol{z}) = \Psi_*(\boldsymbol{z}) \\ A = WD, \quad B = -W, \quad \boldsymbol{c} = \boldsymbol{0} \end{aligned} \tag{23}$$

The parameter $\boldsymbol{z}$ is an unknown variable that satisfies $\boldsymbol{z} = D\boldsymbol{x}$ upon convergence. Following the arguments presented in [41], variable $\boldsymbol{z}$ is subdivided locally into blocks of the same size corresponding to D_i. Likewise, W is a block diagonal matrix.

$$\begin{aligned} \boldsymbol{z} &= \begin{bmatrix} \boldsymbol{z}_1 & \boldsymbol{z}_2 & \dots & \boldsymbol{z}_m \end{bmatrix}^\top \\ \boldsymbol{W} &= \text{block-diag} \begin{bmatrix} \boldsymbol{W}_1 & \boldsymbol{W}_2 & \dots & \boldsymbol{W}_m \end{bmatrix} \end{aligned} \tag{24}$$

Finally, the potential energy and the constraint are reformulated as

$$\Psi_*(\boldsymbol{z}) = \sum_{i=1}^{m} \Psi_i(\boldsymbol{z}_i), \quad W_i(D_i\boldsymbol{x} - \boldsymbol{z}_i) = 0 \tag{25}$$

Applying the update rules from Equation 20, we obtain

$$\begin{aligned}
\boldsymbol{x}_{k+1} &= \underset{\boldsymbol{x}}{\operatorname{argmin}} \ \frac{1}{2\Delta t^2}||\boldsymbol{x}-\tilde{\boldsymbol{x}}||_M^2 + \frac{\rho}{2}||W(D\boldsymbol{x}-\boldsymbol{z}_k+\tilde{\boldsymbol{\lambda}}_k)||^2 \\
\boldsymbol{z}_{k+1} &= \underset{\boldsymbol{z}}{\operatorname{argmin}} \ \Psi_*(\boldsymbol{z}) + \frac{\rho}{2}||W(D\boldsymbol{x}_{k+1}-\boldsymbol{z}+\tilde{\boldsymbol{\lambda}}_k)||^2 \\
\tilde{\boldsymbol{\lambda}}_{k+1} &= \tilde{\boldsymbol{\lambda}}_k + H(\boldsymbol{x}_{k+1},\boldsymbol{z}_{k+1},\boldsymbol{\lambda}_k)^{-1}(D\boldsymbol{x}_{k+1}-\boldsymbol{z}_{k+1})
\end{aligned} \tag{26}$$

where $\tilde{\boldsymbol{\lambda}} = \boldsymbol{\lambda}/\rho$ is a scaled dual variable [6]. This form of ADMM is useful for computation. At convergence, i.e. at $\boldsymbol{x} = \boldsymbol{x}^*$, the corresponding local coordinates satisfy $\boldsymbol{z}^* = D\boldsymbol{x}^*$.

3.4.1 Approximation for Hyperelastic Material. However, applying the proposed method directly to deformable object simulation with a hyperelastic potential energy suffers from a significant drawback. The primary runtime advantage of ADMM over Newton's method is that, unlike Newton's method (Equation 4), ADMM does not require computing the exact inverse of the Hessian at each iteration. As noted in [52] [22], the most time-consuming component of the simulation is the calculation of the inverse Hessian. However, as evident from Equation 26, the augmented Newton update step eliminates that advantage. In that update step, the term $H(\boldsymbol{x},\boldsymbol{z},\boldsymbol{\lambda})$ contains the Hessian of the hyperelastic potential term $\nabla^2_{\boldsymbol{zz}} g(\boldsymbol{z}) = \nabla^2_{\boldsymbol{zz}}\Psi_*(\boldsymbol{z})$, which needs to be recalculated at each iteration for most of the hyperelastic energies.

To address the problem, we approximate the energy Hessian as constant over the simulation time. Let's have a closer look at each of the terms of the Hessian matrix, $H(\boldsymbol{x},\boldsymbol{z},\boldsymbol{\lambda})$.

$$\begin{aligned}
&A = WD, \quad B = -W, \qquad (\textbf{\textit{constants}}) \\
&\nabla^2_{\boldsymbol{xx}} f(\boldsymbol{x}) = \nabla^2_{\boldsymbol{xx}}\left(\frac{1}{2\Delta t^2}||\boldsymbol{x}-\tilde{\boldsymbol{x}}||_M^2\right) = \frac{M}{\Delta t^2}, \qquad (\textbf{\textit{constant}})
\end{aligned} \tag{27}$$

Now, for linear energy of the form,

$$\Psi_*(\boldsymbol{z}) = \frac{1}{2}k||\boldsymbol{z}-\mathbb{1}||_2^2, \quad \exists k \text{ such that } k>0 \text{ and constant}, \tag{28}$$

The Hessian of the energy is computed as follows

$$\nabla^2_{\boldsymbol{zz}} g(\boldsymbol{z}) = \nabla^2_{\boldsymbol{zz}}\Psi_*(\boldsymbol{z}) = k, \qquad (\textbf{\textit{constant}}) \tag{29}$$

However, for other hyperelastic energy densities such as StVK and Neo-Hookean, the energy Hessian does not yield a constant value. Thus, we approximate Hessians for StVK & Neo-Hookean energy densities as follows

$$\nabla^2_{\boldsymbol{zz}} g(\boldsymbol{z}) = \nabla^2_{\boldsymbol{zz}}\Psi_*(\boldsymbol{z}) = (2\mu_{\text{Lamé}} + \lambda_{\text{Lamé}})\, I, \qquad (\textbf{\textit{constant}}) \tag{30}$$

where I denotes the identity matrix. The details of the derivation of $\nabla^2_{\boldsymbol{zz}} g(\boldsymbol{z})$ for both StVK and Neo-Hookean energy density are given in the supplementary document. With the approximation from Equation 30, all the components of $H(\boldsymbol{x}_{k+1},\boldsymbol{z}_{k+1},\boldsymbol{\lambda}_k)$ from Equation 21 remain constant over the course of each individual simulation. Thus, this approach addresses the hurdle of high computational cost.

3.5 Convergence Analysis

Suppose there is a solution $(\boldsymbol{x}^*,\boldsymbol{z}^*,\boldsymbol{\lambda}^*)$ and define $\boldsymbol{e}_k = \boldsymbol{\lambda}_k - \boldsymbol{\lambda}^*$. Near optimality, Taylor expansion of the dual gradient

$$\begin{aligned}
&\nabla_{\boldsymbol{\lambda}} d(\boldsymbol{\lambda}_k) = \nabla^2_{\boldsymbol{\lambda\lambda}} d(\boldsymbol{\lambda}^*)\,\boldsymbol{e}_k + O\left(\|\boldsymbol{e}_k\|^2\right) \\
&\equiv \boldsymbol{r}_k = -H^*\boldsymbol{e}_k + O\left(\|\boldsymbol{e}_k\|^2\right)
\end{aligned} \tag{31}$$

Considering with Hessian approximation $\widehat{H}_k$, using Equation 20

$$\begin{aligned}
&\boldsymbol{\lambda}_{k+1} = \boldsymbol{\lambda}_k + \widehat{H}_k^{-1}\boldsymbol{r}_k \\
&\equiv \boldsymbol{e}_{k+1} = \left(I - \widehat{H}_k^{-1}H^*\right)\boldsymbol{e}_k + O\left(\|\boldsymbol{e}_k\|^2\right)
\end{aligned} \tag{32}$$

Therefore, ADMM augmented with the Newton step is locally linearly convergent if $r_s\left(I - \widehat{H}_k^{-1}H^*\right) < 1$ where r_s denotes the spectral radius.

3.5.1 Full Newton ADMM with dynamic Hessian. Suppose $\widehat{H} = H^*$, then $\boldsymbol{e}_{k+1} = O\left(\|\boldsymbol{e}_k\|^2\right)$, making the convergence quadratic.

3.5.2 Quasi Newton ADMM with Fixed Hessian. However, in our case, for deformable object simulation, we assumed a fixed Hessian rather than updating it at each time step as

$$\widehat{H}_c = H^* + E \tag{33}$$

where E is the Hessian approximation error. Let $\mathcal{T} = \left(I - \widehat{H}_c^{-1}H^*\right)$

$$\begin{aligned}
\mathcal{T} &= I - \widehat{H}_c^{-1}H^* = I - (H^*+E)^{-1}H^* \\
&= I - \left(I + {H^*}^{-1}E\right)^{-1} = \left(I + {H^*}^{-1}E\right)^{-1}{H^*}^{-1}E
\end{aligned} \tag{34}$$

Expanding using the Neumann series and taking the norm on both sides, after some algebraic manipulation, we get

$$\|\mathcal{T}\| \le \frac{\|{H^*}^{-1}E\|}{1 - \|{H^*}^{-1}E\|} \tag{35}$$

Thus, a sufficient condition for local convergence is

$$\frac{\|{H^*}^{-1}E\|}{1 - \|{H^*}^{-1}E\|} < 1 \quad \equiv \quad \|{H^*}^{-1}E\| < 0.5 \tag{36}$$

This completes the proof of the local convergence analysis for Newton-augmented ADMM with a fixed Hessian. In the next section, we discuss our implementation and the results obtained.

4 Results and Discussion

We present a range of simulation visualizations across diverse materials to demonstrate the robustness of our method and its ability to handle a wide variety of scenarios. We then compare our approach with existing ADMM and projective dynamics methods to demonstrate its advantages over existing algorithms. All simulations were conducted on an Intel Xeon Gold 6326 CPU with 32 threads running at 2.90 GHz.

4.1 Deformable Object Simulation

In Figure 1, we simulate multiple deformable bodies using ADMM augmented with a Newton step, demonstrating the method's robustness in handling complex contact dynamics. The approach reliably supports large deformations, heterogeneous material properties, and simultaneous object–object, object–surface, and self-collisions.



Figure 6: Signorini Test: A ball falls onto a rigid sheet and rebounds.



Figure 7: A low-stiffness Saint Venant-Kirchhoff bunny recovers from having its vertices randomized.

In the first scenario (top), six box-shaped objects with linear hyperelastic energies are dropped onto a rigid surface, where they undergo repeated impacts, mutual collisions, and self-contact. In the second scenario (bottom), two tori and two boxes with different elastic properties are dropped onto a rigid sheet, rebound, and interact through multiple collisions, further illustrating the method's stability and effectiveness in challenging multi-contact simulations.

Figure 3 illustrates the uniaxial stretching of three beams, each modeled using a different hyperelastic energy formulation. From top to bottom, the beams use Neo-Hookean, Saint Venant–Kirchhoff (StVK), and co-rotational material models, respectively. Each beam contains 540 tetrahedral elements and 208 vertices. Although all three beams are subjected to identical boundary conditions and loading, their deformation responses differ, highlighting the influence of the chosen strain-energy density on large-deformation behavior.

Figure 4 illustrates two boxes that fall freely under the influence of gravity onto a deformable sheet. Each box consists of 1175 tetrahedral elements and 413 vertices. After impact, the boxes come to rest stacked on top of one another, demonstrating the interaction between rigid bodies and the supporting surface under gravitational loading. A dynamic contact experiment is shown in Figure 5, in which a deformable torus falls under gravity onto a rigid sheet and subsequently rebounds. Torus mesh consists of 4130 tetrahedral elements and 798 vertices. The simulation demonstrates the method's ability to handle large deformations, self-contact response, and elastic recovery in three dimensions.

Figure 6 depicts a classical Signorini contact test, where a deformable ball impacts a rigid sheet and rebounds. The deformable sphere consists of 3100 tetrahedral elements and 957 vertices. This example validates the robustness of the contact handling, enforcing non-penetration while allowing separation after impact and preserving the expected qualitative dynamics of the system. A low-stiffness Saint Venant-Kirchhoff bunny recovers from having its vertices randomized in Figure 7. This experiment shows the robustness of our method. The bunny mesh consists of 2510 tetrahedral elements and 777 vertices.

4.2 Comparison Study

We compared our proposed algorithm with the state-of-the-art ADMM technique [41] in terms of simulation runtime and the number of iterations required for convergence. The detailed comparison data are presented in Table 1. As evident from the tabular data, our method runs $1.2 - 1.9\times$ faster than the standard ADMM-based technique [41]. Moreover, our proposed method requires fewer iterations compared to the existing algorithm. As shown in Table 1, our method converges faster than the existing algorithm in fewer iterations. The fractional iteration counts reported in the table represent the average number of iterations required by the solvers. In all of our experiments, we use the stopping criterion that the residual falls below 10^{-6}. As further validation, Figure 8 compares the convergence behavior of standard ADMM and ADMM augmented with a Newton step across two scenarios: beam and torus. The results clearly show that the proposed method converges more quickly, requiring significantly fewer iterations to achieve the same level of accuracy. This improvement highlights the effectiveness of incorporating the Newton step for accelerating convergence in the ADMM framework.

5 Limitation and Future Work

We propose a novel ADMM-based optimization algorithm augmented with Newton steps. We further adapt this technique to address the implicit time-integration problem arising in deformable-object simulation. Through a range of experimental evaluations, we demonstrate that our method converges rapidly and consistently outperforms existing approaches [41] in computational efficiency and simulation runtime.

Despite these advantages, our method has certain limitations. In particular, during the Newton step, the Hessian matrix is not computed explicitly. Instead, we employ an approximation. While this approximation improves computational efficiency, faster runtime, and accuracy compared to the ADMM [41] method, it yields identical Hessian expressions across distinct hyperelastic energy models, potentially limiting the method's ability to fully capture material-specific behavior. This leads to lower accuracy than state-of-the-art FEM methods [22] [53]. Addressing this limitation by incorporating more accurate or energy-specific Hessian formulations may be a promising avenue for future work.

References

[1] David Baraff and Andrew Witkin. 1998. Large steps in cloth simulation. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '98)*. Association for Computing Machinery, New York, NY, USA, 43–54.

[2] Jan Bender, Matthias Müller, Miguel A. Otaduy, Matthias Teschner, and Miles Macklin. 2014. A Survey on Position-Based Simulation Methods in Computer Graphics. *Computer Graphics Forum* 33, 6 (2014), 228–251.

[3] Bernd Bickel, Moritz Bächer, Miguel A. Otaduy, Wojciech Matusik, Hanspeter Pfister, and Markus Gross. 2009. Capture and modeling of non-linear heterogeneous soft tissue. *ACM Trans. Graph.* 28, 3, Article 89 (July 2009), 9 pages. https://doi.org/10.1145/1531326.1531395

[4] Sofien Bouaziz, Mario Deuss, Yuliy Schwartzburg, Thibaut Weise, and Mark Pauly. 2012. Shape-Up: Shaping Discrete Geometry with Projections. *Computer Graphics Forum* 31, 5 (2012), 1657–1667.

[5] Sofien Bouaziz, Sebastian Martin, Tiantian Liu, Ladislav Kavan, and Mark Pauly. 2014. Projective dynamics: fusing constraint projections for fast simulation. *ACM Trans. Graph.* 33, 4, Article 154 (July 2014), 11 pages.

[6] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. 2011. Distributed Optimization and Statistical Learning via the Alternating Direction

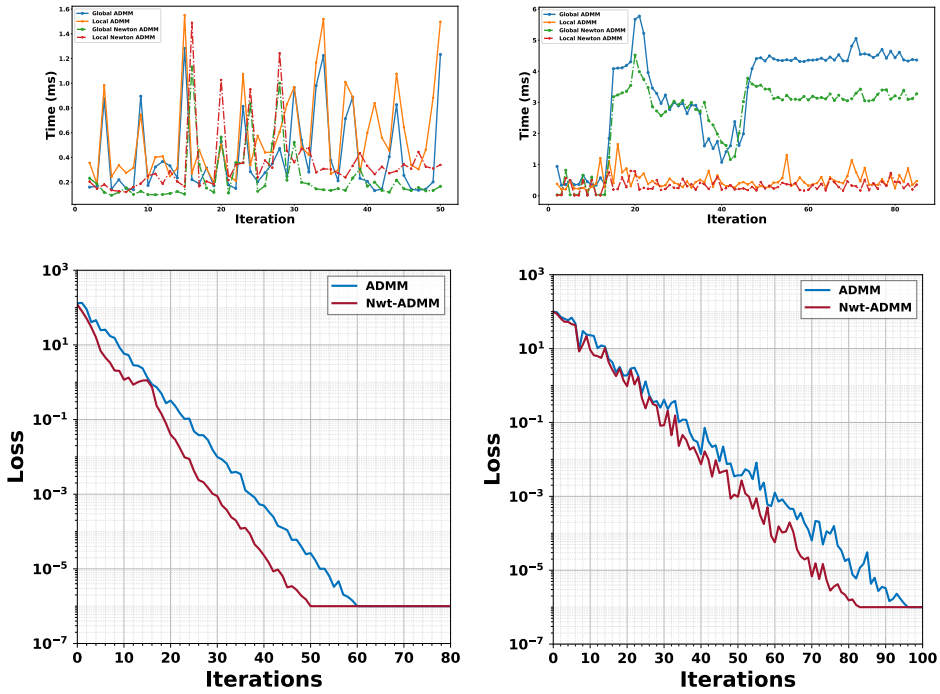


Figure 8: A comparison of the convergence behavior of standard ADMM and ADMM augmented with a Newton step for beam (left col.) and torus (right col.) scenes.

Table 1: Newton ADMM vs ADMM: T.G., T.L., A.G., and A.L. denote the total global, total local, average global, and average local optimization times, respectively, measured in milliseconds. Total Local time is split into z and λ update as T.L.(z) and T.L.(λ) respectively. Additionally, we reported the average and total number of iterations taken by the solvers.

Experiment	T.G.	T.L.	T.L. (z)	T.L. (λ)	A.G.	A.L.	# Itr. (Avg.)	# Itr. (Tot.)
Beam ***[Ours]***	4.77	7.11	6.04	1.07	0.238	0.355	0.89	51
Beam [41]	8.05	11.88	10.1	1.78	0.402	0.594	1	58
Torus ***[Ours]***	32.76	4.50	3.7	0.8	3.27	0.451	15.6	86
Torus [41]	37.87	5.23	4.29	0.94	3.79	0.523	16.98	94
Signorini ***[Ours]***	20.10	2.25	1.98	0.27	2.01	0.226	15.19	35
Signorini [41]	29.97	2.37	2.13	0.24	2.99	0.24	30	69
Box ***[Ours]***	33.74	1.38	1.2	0.18	3.37	0.138	22.6	72
Box [41]	38.81	1.95	1.69	0.26	3.88	0.19	28.47	91

Method of Multipliers. *Foundations and Trends in Machine Learning* 3, 1 (2011), 1–122.

[7] Anka He Chen, Ziheng Liu, Yin Yang, and Cem Yuksel. 2024. Vertex Block Descent. *ACM Trans. Graph.* 43, 4, Article 116 (July 2024), 16 pages.

[8] Gilles Daviet. 2023. Interactive Hair Simulation on the GPU using ADMM. In *ACM SIGGRAPH 2023 Conference Proceedings* (Los Angeles, CA, USA) *(SIGGRAPH '23)*. Association for Computing Machinery, New York, NY, USA, Article 24, 11 pages. https://doi.org/10.1145/3588432.3591551

[9] Chih-Hao Fang, Sudhir B. Kylasa, Fred Roosta, Michael W. Mahoney, and Ananth Grama. 2020. Newton-ADMM: A Distributed GPU-Accelerated Optimizer for Multiclass Classification Problems. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–12.

[10] Pedro A. Forero, Alfonso Cano, and Georgios B. Giannakis. 2011. Distributed Clustering Using Wireless Sensor Networks. *IEEE Journal of Selected Topics in Signal Processing* 5, 4 (2011), 707–724.

[11] Marco Fratarcangeli, Valentina Tibaldo, and Fabio Pellacini. 2016. Vivace: a practical gauss-seidel method for stable soft body dynamics. *ACM Trans. Graph.* 35, 6, Article 214 (Dec. 2016), 9 pages.

[12] T. F. Gast and C. Schroeder. 2015. Optimization integrator for large time steps. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation* (Copenhagen, Denmark) *(SCA '14)*. Eurographics Association, Goslar, DEU, 31–40.

[13] Philip E. Gill, Walter Murray, and Margaret H. Wright. 1981. *Practical optimization*. Academic Press, London.

[14] Donald Goldfarb, Shiqian Ma, and Katya Scheinberg. 2013. Fast alternating linearization methods for minimizing the sum of two convex functions. *Mathematical Programming* 141, 1 (2013), 349–382.

[15] Tom Goldstein, Brendan O'Donoghue, Simon Setzer, and Richard Baraniuk. 2014. Fast Alternating Direction Optimization Methods. *SIAM Journal on Imaging Sciences* 7, 3 (2014), 1588–1623.

[16] James Gregson, Ivo Ihrke, Nils Thuerey, and Wolfgang Heidrich. 2014. From capture to simulation: connecting forward and inverse problems in fluids. *ACM Trans. Graph.* 33, 4, Article 139 (July 2014), 11 pages.

[17] Mingyi Hong, Zhi-Quan Luo, and Meisam Razaviyayn. 2015. Convergence analysis of alternating direction method of multipliers for a family of nonconvex problems. In *2015 IEEE International Conference on Acoustics, Speech and Signal*

Processing (ICASSP). 3836–3840.
[18] Mojtaba Kadkhodaie, Konstantina Christakopoulou, Maziar Sanjabi, and Arindam Banerjee. 2015. Accelerated Alternating Direction Method of Multipliers. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Sydney, NSW, Australia) *(KDD '15)*. Association for Computing Machinery, New York, NY, USA, 497–506.
[19] Min Hyung Kee, Kiwon Um, Wooseok Jeong, and Junghyun Han. 2021. Constrained projective dynamics: real-time simulation of deformable objects with energy-momentum conservation. *ACM Trans. Graph.* 40, 4, Article 160 (July 2021), 12 pages.
[20] L. Kharevych, Weiwei Yang, Y. Tong, E. Kanso, J. E. Marsden, P. Schröder, and M. Desbrun. 2006. Geometric, variational integrators for computer animation. In *Proceedings of the 2006 ACM SIGGRAPH/Eurographics Symposium on Computer Animation* (Vienna, Austria) *(SCA '06)*. Eurographics Association, Goslar, DEU, 43–51.
[21] Theodore Kim, Fernando De Goes, and Hayley Iben. 2019. Anisotropic elasticity for inversion-safety and element rehabilitation. *ACM Trans. Graph.* 38, 4, Article 69 (July 2019), 15 pages.
[22] Theodore Kim and David Eberle. 2022. Dynamic deformables: implementation and production practicalities (now with code!). In *ACM SIGGRAPH 2022 Courses* (Vancouver, British Columbia, Canada) *(SIGGRAPH '22)*. Association for Computing Machinery, New York, NY, USA, Article 7, 259 pages.
[23] Lei Lan, Minchen Li, Chenfanfu Jiang, Huamin Wang, and Yin Yang. 2023. Second-order Stencil Descent for Interior-point Hyperelasticity. *ACM Trans. Graph.* 42, 4, Article 108 (July 2023), 16 pages.
[24] Guoyin Li and Ting Kei Pong. 2015. Global Convergence of Splitting Methods for Nonconvex Composite Optimization. *SIAM Journal on Optimization* 25, 4 (2015), 2434–2460.
[25] Jing Li, Tiantian Liu, Ladislav Kavan, and Baoquan Chen. 2021. Interactive cutting and tearing in projective dynamics with progressive cholesky updates. *ACM Trans. Graph.* 40, 6, Article 254 (Dec. 2021), 12 pages.
[26] Minchen Li, Ming Gao, Timothy Langlois, Chenfanfu Jiang, and Danny M. Kaufman. 2019. Decomposed optimization time integrator for large-step elastodynamics. *ACM Trans. Graph.* 38, 4, Article 70 (July 2019), 10 pages.
[27] Yichuan Li, Nikolaos M. Freris, Petros Voulgaris, and Dušan Stipanović. 2021. DN-ADMM: Distributed Newton ADMM for Multi-agent Optimization. In *2021 60th IEEE Conference on Decision and Control (CDC)*. 3343–3348.
[28] Tiantian Liu, Adam W. Bargteil, James F. O'Brien, and Ladislav Kavan. 2013. Fast simulation of mass-spring systems. *ACM Trans. Graph.* 32, 6, Article 214 (Nov. 2013), 7 pages.
[29] Tiantian Liu, Sofien Bouaziz, and Ladislav Kavan. 2017. Quasi-Newton Methods for Real-Time Simulation of Hyperelastic Materials. *ACM Trans. Graph.* 36, 4, Article 116a (July 2017), 16 pages.
[30] Jia-Ming Lu and Shi-Min Hu. 2025. Reliable Iterative Dynamics: A Versatile Method for Fast and Robust Simulation. *ACM Trans. Graph.* 44, 3, Article 29 (May 2025), 18 pages.
[31] Jia-Ming Lu, Tailing Yuan, Zhe-Han Mo, and Shi-Min Hu. 2025. Fast Galerkin Multigrid Method for Unstructured Meshes. *ACM Trans. Graph.* 44, 6, Article 179 (Dec. 2025), 16 pages.
[32] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. 2016. XPBD: position-based simulation of compliant constrained dynamics. In *Proceedings of the 9th International Conference on Motion in Games* (Burlingame, California) *(MIG '16)*. Association for Computing Machinery, New York, NY, USA, 49–54.
[33] Sindri Magnússon, Pradeep Chathuranga Weeraddana, Michael G. Rabbat, and Carlo Fischione. 2016. On the Convergence of Alternating Direction Lagrangian Methods for Nonconvex Structured Optimization Problems. *IEEE Transactions on Control of Network Systems* 3, 3 (2016), 296–309.
[34] Sebastian Martin, Bernhard Thomaszewski, Eitan Grinspun, and Markus Gross. 2011. Example-based elastic materials. In *ACM SIGGRAPH 2011 Papers* (Vancouver, British Columbia, Canada) *(SIGGRAPH '11)*. Association for Computing Machinery, New York, NY, USA, Article 72, 8 pages.
[35] E. Miguel, D. Bradley, B. Thomaszewski, B. Bickel, W. Matusik, M. A. Otaduy, and S. Marschner. 2012. Data-Driven Estimation of Cloth Simulation Models. *Computer Graphics Forum* 31, 2pt2 (2012), 519–528.
[36] Eder Miguel, David Miraut, and Miguel A. Otaduy. 2016. Modeling and Estimation of Energy-Based Hyperelastic Objects. *Computer Graphics Forum* 35, 2 (2016), 385–396.
[37] Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. 2007. Position based dynamics. *J. Vis. Comun. Image Represent.* 18, 2 (April 2007), 109–118.
[38] Rahul Narain, Armin Samii, and James F. O'Brien. 2012. Adaptive anisotropic remeshing for cloth simulation. *ACM Trans. Graph.* 31, 6, Article 152 (Nov. 2012), 10 pages.
[39] Jorge Nocedal and Stephen J. Wright. 2006. *Numerical Optimization* (2 ed.). Springer, New York.
[40] Wenqing Ouyang, Yue Peng, Yuxin Yao, Juyong Zhang, and Bailin Deng. 2020. Anderson Acceleration for Nonconvex ADMM Based on Douglas-Rachford Splitting. *Computer Graphics Forum* 39, 5 (2020), 221–239.
[41] Matthew Overby, George E. Brown, Jie Li, and Rahul Narain. 2017. ADMM ⊇ Projective Dynamics: Fast Simulation of Hyperelastic Models with Dynamic Constraints. *IEEE Transactions on Visualization and Computer Graphics* 23, 10 (2017), 2222–2234.
[42] Matthew Overby, Danny Kaufman, and Rahul Narain. 2021. Globally Injective Geometry Optimization with Non-Injective Steps. *Computer Graphics Forum* 40, 5 (2021), 111–123.
[43] Breannan Smith, Fernando De Goes, and Theodore Kim. 2018. Stable Neo-Hookean Flesh Simulation. *ACM Trans. Graph.* 37, 2, Article 12 (March 2018), 15 pages.
[44] Breannan Smith, Fernando De Goes, and Theodore Kim. 2019. Analytic Eigensystems for Isotropic Distortion Energies. *ACM Trans. Graph.* 38, 1, Article 3 (Feb. 2019), 15 pages.
[45] Dennis L. Sun and Cédric Févotte. 2014. Alternating direction method of multipliers for non-negative matrix factorization with the beta-divergence. In *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 6201–6205.
[46] Demetri Terzopoulos, John Platt, Alan Barr, and Kurt Fleischer. 1987. Elastically deformable models. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '87)*. Association for Computing Machinery, New York, NY, USA, 205–214.
[47] Paul Tseng. 1991. Applications of a Splitting Algorithm to Decomposition in Convex Programming and Variational Inequalities. *SIAM Journal on Control and Optimization* 29, 1 (1991), 119–138.
[48] Huamin Wang. 2015. A chebyshev semi-iterative approach for accelerating projective and position-based dynamics. *ACM Trans. Graph.* 34, 6, Article 246 (Nov. 2015), 9 pages.
[49] Huamin Wang, James F. O'Brien, and Ravi Ramamoorthi. 2011. Data-driven elastic models for cloth: modeling and measurement. *ACM Trans. Graph.* 30, 4, Article 71 (July 2011), 12 pages.
[50] Marcel Weiler, Dan Koschier, and Jan Bender. 2016. Projective fluids. In *Proceedings of the 9th International Conference on Motion in Games* (Burlingame, California) *(MIG '16)*. Association for Computing Machinery, New York, NY, USA, 79–84.
[51] Zaiwen Wen, Chao Yang, Xin Liu, and Stefano Marchesini. 2012. Alternating direction methods for classical and ptychographic phase retrieval. *Inverse Problems* 28, 11 (oct 2012), 115010.
[52] Zangyueyang Xian, Xin Tong, and Tiantian Liu. 2019. A scalable galerkin multigrid method for real-time simulation of deformable objects. *ACM Trans. Graph.* 38, 6, Article 162 (Nov. 2019), 13 pages.
[53] Hongyi Xu, Funshing Sin, Yufeng Zhu, and Jernej Barbič. 2015. Nonlinear material design using principal stretches. *ACM Trans. Graph.* 34, 4, Article 75 (July 2015), 11 pages.
[54] Juyong Zhang, Yue Peng, Wenqing Ouyang, and Bailin Deng. 2019. Accelerating ADMM for efficient simulation and optimization. *ACM Trans. Graph.* 38, 6, Article 163 (Nov. 2019), 21 pages.
[55] Mingqiang Zhu and Tony F. Chan. 2008. An Efficient Primal-Dual Hybrid Gradient Algorithm For Total Variation Image Restoration. https://api.semanticscholar.org/CorpusID:11110283

A Supplementary Materials

Supplementary materials are shared at the following link:
https://github.com/avirupmandal/Supp-Newton-ADMM.git